

Nginx: One Website Served on Multiple Ports (Advanced Setup)

This is not the “two separate sites” pattern — this is **one codebase, one document root, exposed on several ports at once**, with practical extras: per-port access logging, a health-check endpoint, a port-based redirect rule, gzip, security headers, and a systemd-aware reload workflow.

Ports used in this guide (change to whatever you need):

Port	Purpose
8081	Primary / public
8082	Internal / staging mirror of the same site
8083	Health-check & monitoring port (restricted)

1. Install Nginx

Ubuntu/Debian

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable --now nginx
```

CentOS/RHEL/Fedora

```
sudo dnf install nginx -y
sudo systemctl enable --now nginx
```

Confirm version and modules compiled in (useful for gzip/headers later):

```
nginx -V
```

2. Create the Site Content

```
sudo mkdir -p /var/www/mysite
sudo tee /var/www/mysite/index.html > /dev/null << 'EOF'
<!DOCTYPE html>
<html>
<head><title>My Website</title></head>
<body>
  <h1>My Website</h1>
  <p>Served on port: <span id="port"></span></p>
  <script>
    document.getElementById('port').textContent =
window.location.port || '80';
  </script>
</body>
</html>
EOF
```

```

sudo mkdir -p /var/www/mysite/health
echo "OK" | sudo tee /var/www/mysite/health/index.html

sudo chown -R www-data:www-data /var/www/mysite # Ubuntu/Debian
# sudo chown -R nginx:nginx /var/www/mysite      # CentOS/RHEL

```

The little inline script above shows which port served the page — handy for confirming multi-port setups actually work, not just that a page loaded.

3. The Advanced Server Block

This single config exposes the same root on three ports, but each port behaves slightly differently: 8083 only serves /health and denies everything else, and requests to 8082 get an X-Served-Port response header for debugging.

Ubuntu/Debian: /etc/nginx/sites-available/mysite **CentOS/RHEL:** /etc/nginx/conf.d/mysite.conf

```

# Custom log format that records which port handled the request
log_format multiport '$remote_addr - $server_port [$time_local] '
                    '$request' $status $body_bytes_sent '
                    '$http_referer' '$http_user_agent'
rt=$request_time';

server {
    listen 8081;
    listen 8082;
    server_name localhost;
    root /var/www/mysite;
    index index.html;

    access_log /var/log/nginx/mysite_access.log multiport;
    error_log /var/log/nginx/mysite_error.log warn;

    # Basic security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Served-Port $server_port always;

    # Gzip compression
    gzip on;
    gzip_types text/plain text/css application/javascript
application/json text/xml;
    gzip_min_length 256;

    location / {
        try_files $uri $uri/ =404;
    }

    # Deny access to hidden files (.env, .git, etc.)
    location ~ /\. {
        deny all;
    }
}

# Dedicated health-check port – only serves /health, blocks

```

```

everything else,
# and is typically firewalled to internal/monitoring IPs only.
server {
    listen 8083;
    server_name localhost;
    root /var/www/mysite;

    access_log off;    # health checks are noisy – keep them out of
the main log

    location = /health {
        default_type text/plain;
        try_files /health/index.html =503;
    }

    location / {
        return 404;
    }
}

```

Why this is more than “just add another listen line”

- **Per-port log format** (\$server_port in log_format) lets you tell 8081 vs 8082 traffic apart in one shared log file.
- **X-Served-Port header** makes it trivial to confirm with `curl -I` exactly which listener answered.
- **Separate health-check server block** on 8083 avoids mixing monitoring probes into your real traffic logs, and gives you a port you can expose only to a load balancer or monitoring tool without exposing the whole site.
- **Security headers + gzip** are the kind of production details a “just make it work” config skips.

4. Enable, Test, Reload

```

# Ubuntu/Debian only
sudo ln -s /etc/nginx/sites-available/mysite /etc/nginx/sites-
enabled/

sudo nginx -t
sudo systemctl reload nginx

```

5. Firewall

```

# UFW
sudo ufw allow 8081/tcp
sudo ufw allow 8082/tcp
sudo ufw allow from 10.0.0.0/8 to any port 8083 proto tcp    #
restrict health port to internal network

# firewalld
sudo firewall-cmd --permanent --add-port=8081/tcp
sudo firewall-cmd --permanent --add-port=8082/tcp
sudo firewall-cmd --permanent --zone=internal --add-port=8083/tcp
sudo firewall-cmd --reload

```

SELinux (CentOS/RHEL, if enforcing):

```
sudo semanage port -a -t http_port_t -p tcp 8081
sudo semanage port -a -t http_port_t -p tcp 8082
sudo semanage port -a -t http_port_t -p tcp 8083
```

6. Verify Everything

```
# Confirm content is identical on both public ports
curl -s http://localhost:8081 | md5sum
curl -s http://localhost:8082 | md5sum

# Confirm the debug header shows the right port
curl -I http://localhost:8081 | grep X-Served-Port
curl -I http://localhost:8082 | grep X-Served-Port

# Confirm health port only serves /health
curl http://localhost:8083/health # → OK
curl -i http://localhost:8083/    # → 404

# Watch which port each request came in on
sudo tail -f /var/log/nginx/mysite_access.log
```

7. Optional: Redirect One Port to Another

If you later decide port 8082 should just forward to 8081 instead of serving its own copy:

```
server {
    listen 8082;
    server_name localhost;
    return 301 http://$host:8081$request_uri;
}
```

8. Optional: Reload Automatically on Config Change

```
sudo apt install inotify-tools -y # Ubuntu/Debian
while inotifywait -e modify /etc/nginx/sites-available/mysite; do
    sudo nginx -t && sudo systemctl reload nginx
done
```

Useful in dev environments only — don't run this in production.

Directory & File Summary

/var/www/mysite/index.html	→ shared content, served on 8081 and 8082
/var/www/mysite/health/index.html	→ health check content, served on 8083 only
/etc/nginx/sites-available/mysite	→ config (Ubuntu/Debian)

<code>/etc/nginx/conf.d/mysite.conf</code>	→ config (CentOS/RHEL)
<code>/var/log/nginx/mysite_access.log</code>	→ combined access log for
<code>8081 + 8082, tagged by \$server_port</code>	
<code>/var/log/nginx/mysite_error.log</code>	→ error log

See `TROUBLESHOOTING.md` for diagnosing issues specific to multi-port single-site setups, and `STEPS.md` for the condensed step list.